Java Assignment 1 – Slot C

Name: T. Ganesh(192425246) CSA0909

Section A: Multiple-Choice Questions

1. Division and Widening

```
int a = 10;
double d = a / 4;
System.out.println(d);
```

A) 2.5
B) 2.0
C) 2
D) 3.0

Answer: B) 2.0

Explanation: Both a and 4 are int, so a / 4 is integer division: 10 / 4 = 2 (the .5 is discarded). Only after that is the int result 2 widened to double and stored in d, giving 2.0. The division happens before the widening, so the decimal part is already lost.

2. Output and Concatenation

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

A) 12345
B) 15
C) 3345
D) Compilation error

Answer: C) 3345

Explanation: The + operator is evaluated left to right. First 1 + 2 = 3 (both are numbers). Then 3 + "3" becomes the String "33". From this

point on, everything is String concatenation: "33" + 4 = "334", then "334" + 5 = "3345".

3. Post-Increment and Pre-Increment Together

```
int a = 3;
int b = a++ + ++a * a--;
System.out.println(a + " " + b);
```

A) 4 28
B) 3 28
C) 5 28
D) 4 26

Answer: A) 4 28

Explanation: Operands are evaluated left to right, and * has higher precedence than +, so the expression is a++ + (++a * a--). Trace:

Step	Operation	Value used	a after
1	a++ (post-increment)	3	4
2	++a (pre-increment)	5	5
3	a-- (post-decrement)	5	4

So b = 3 + (5 * 5) = 3 + 25 = 28, and the final value of a is 4. Output: **4 28**

4. The else-if Ladder

```
int m = 72;
if (m >= 90)
    System.out.println("Gold");
else if (m >= 60)
    System.out.println("Silver");
else if (m >= 70)
    System.out.println("Bronze");
else
```

```
    System.out.println("None");
```

A) Gold
B) Bronze
C) None
D) Silver

Answer: D) Silver

Explanation: Conditions are checked from top to bottom and the first true one wins. 72 >= 90 is false; 72 >= 60 is true, so "Silver" is printed and the rest of the ladder is skipped. The m >= 70 (Bronze) branch is never reached because the m >= 60 check above it already catches every value from 60 upward.

5. default in the Middle of a switch

```
int n = 3;
switch (n) {
    case 1: System.out.print("One ");
    default: System.out.print("Many ");
    case 2: System.out.print("Two "); break;
    case 4: System.out.print("Four ");
}
```

A) Many
B) Many Two
C) Two
D) Many Two Four

Answer: B) Many Two

Explanation: n = 3 matches no case label, so control jumps to default (even though it sits in the middle). "Many " is printed. default has no break, so execution falls through into case 2 and prints "Two ". The break there stops the switch, so case 4 is never executed.

6. Identify the Error

```
int a = 5;
if (a = 5) {
    System.out.println("Five");
}
```

A) Five
B) Nothing is printed
C) Compilation error
D) Runtime exception

Answer: C) Compilation error

Explanation: a = 5 is an assignment, not a comparison, and its result is the int value 5. Unlike C/C++, Java's if requires a boolean expression, so the compiler reports "incompatible types: int cannot be converted to boolean". The correct condition is a == 5.

7. Short-Circuit ||

```
int a = 6, b = 0;
boolean r = (a > 5) || (a / b > 1);
System.out.println(r);
```

A) true
B) false
C) ArithmeticException at runtime
D) Compilation error

Answer: A) true

Explanation: The || operator short-circuits: if the left side is true, the right side is never evaluated. Here a > 5 is true (6 > 5), so (a / b > 1) is skipped and the division by zero never happens. r becomes true. (With the non-short-circuit | operator, an ArithmeticException would occur.)

8. Literals

```
long big = 3000000000;
System.out.println(big);
```

A) 3000000000
B) -1294967296
C) 3.0E9
D) Compilation error

Answer: D) Compilation error

Explanation: An integer literal without a suffix is of type int, and 3000000000 exceeds the int maximum of 2,147,483,647. The compiler rejects it with "integer number too large" before assignment to long is even considered. The fix is to write it as a long literal: 3000000000L.

9. Nested Ternary

```
int x = 15;
int r = x > 10 ? x < 20 ? x * 2 : x * 3 : x;
System.out.println(r);
```

A) 15
B) 45
C) 30
D) 20

Answer: C) 30

Explanation: The ternary operator groups right to left, so this reads: x > 10 ? (x < 20 ? x * 2 : x * 3) : x. Since 15 > 10 is true, the inner ternary is evaluated. 15 < 20 is true, so the result is x * 2 = 30.

10. Compound Assignment and Precedence

```
int a = 20, b = 6;
a -= b / 2 * 3 + 4 % 3;
System.out.println(a);
```

A) 10
B) 4
C) 12
D) 19

Answer: A) 10

Explanation: A compound assignment evaluates the whole right-hand side first: a = a - (b / 2 * 3 + 4 % 3). Operators /, * and % have equal precedence and go left to right: b / 2 = 3, then 3 * 3 = 9, and 4 % 3 = 1. So the RHS is 9 + 1 = 10, and a = 20 - 10 = 10.

Section B: Scenario-Based Programming Questions

Problem 1: Video Duration Display

Approach

There are 3600 seconds in an hour and 60 seconds in a minute. Integer division and the modulus operator split the total cleanly:

- Hours = total / 3600
- Minutes = (total % 3600) / 60 (seconds left after removing whole hours, divided by 60)
- Seconds = total % 60

Program

```
import java.util.Scanner;

public class VideoDuration {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int total = sc.nextInt();

        int hours   = total / 3600;
        int minutes = (total % 3600) / 60;
        int seconds = total % 60;

        System.out.println("Hours: " + hours);
        System.out.println("Minutes: " + minutes);
        System.out.println("Seconds: " + seconds);
        sc.close();
    }
}
```

Output / Test Results

Input	Hours	Minutes	Seconds	Check
3725	1	2	5	3600 + 120 + 5 = 3725 ✓
59	0	0	59	Matches Sample 2 ✓
0	0	0	0	Lower boundary ✓
86399	23	59	59	Upper boundary ✓

Problem 2: Cinema Ticket Price

Approach

First decide the base price from the age using an if-else-if ladder (order matters, so each range only needs its upper bound). Then, if it is a weekend and the price is greater than 0, add the Rs. 50 surcharge. Free tickets (age below 5) never get the surcharge.

Age	Base price	Weekend price
Below 5	Rs. 0	Rs. 0 (no surcharge on free tickets)
5 – 12	Rs. 100	Rs. 150
13 – 59	Rs. 200	Rs. 250
60 and above	Rs. 120	Rs. 170

Program

```
import java.util.Scanner;

public class CinemaTicket {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int age = sc.nextInt();
        int weekend = sc.nextInt();
```

```
        int price;
        if (age < 5) {
            price = 0;
        } else if (age <= 12) {
            price = 100;
        } else if (age <= 59) {
            price = 200;
        } else {
            price = 120;
        }

        if (weekend == 1 && price > 0) {
            price = price + 50;
        }

        System.out.println("Ticket price: Rs. " + price);
        sc.close();
    }
}
```

Output/ Test Results

Input	Base price	Surcharge	Output
8 1	100	+50	Ticket price: Rs. 150
70 0	120	none (weekday)	Ticket price: Rs. 120
3 1	0	none (free)	Ticket price: Rs. 0
5 0	100	none (weekday)	Ticket price: Rs. 100
13 1	200	+50	Ticket price: Rs. 250
60 1	120	+50	Ticket price: Rs. 170

Problem 3: Personal Loan Eligibility

Approach

The rules must be checked in the given order, and the first failing rule decides the output. An if-else-if ladder does exactly this:

- Age outside 21 – 60 → Not eligible (age)
- Income below 25000 → Not eligible (income)
- Credit score below 650 → Not eligible (credit score)
- Score 650 – 749 → needs no existing loan; maximum loan = 5 × income, otherwise Not eligible (existing loan)
- Score 750 or above → eligible regardless of existing loan; maximum loan = 10 × income

Since the score is already known to be at least 650 once we reach the score checks, the range 650–749 is tested simply as score <= 749, and everything else is 750 or above.

Program

```
import java.util.Scanner;

public class LoanEligibility {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int age = sc.nextInt();
        int income = sc.nextInt();
        int score = sc.nextInt();
        int existingLoan = sc.nextInt();

        if (age < 21 || age > 60) {
            System.out.println("Not eligible (age)");
        } else if (income < 25000) {
            System.out.println("Not eligible (income)");
        } else if (score < 650) {
            System.out.println("Not eligible (credit score)");
        } else if (score <= 749) {
```

```
            if (existingLoan == 1) {
                System.out.println("Not eligible (existing loan)");
            } else {
                System.out.println("Eligible. Maximum loan: " + (5 *
income));
            }
        } else {
            System.out.println("Eligible. Maximum loan: " + (10 *
income));
        }
        sc.close();
    }
}
```

output / Test Results

Input	Deciding rule	Output
30 40000 780 1	Score ≥ 750, loan ignored: 10 × 40000	Eligible. Maximum loan: 400000
35 30000 700 0	Score 650–749, no loan: 5 × 30000	Eligible. Maximum loan: 150000
45 50000 700 1	Score 650–749 but has existing loan	Not eligible (existing loan)
19 30000 800 0	Age below 21	Not eligible (age)
30 20000 800 0	Income below 25000	Not eligible (income)
30 40000 600 0	Score below 650	Not eligible (credit score)
21 25000 650 0	All boundaries pass: 5 × 25000	Eligible. Maximum loan: 125000

Note: All answers and programs above were compiled and run in Java 21 and match the sample outputs in the assignment.